

→ The Mechanism

⇒ Serialization



> each bank serves one thread per cycle,
so conflicts serialize the warp while other banks sit idle during replays

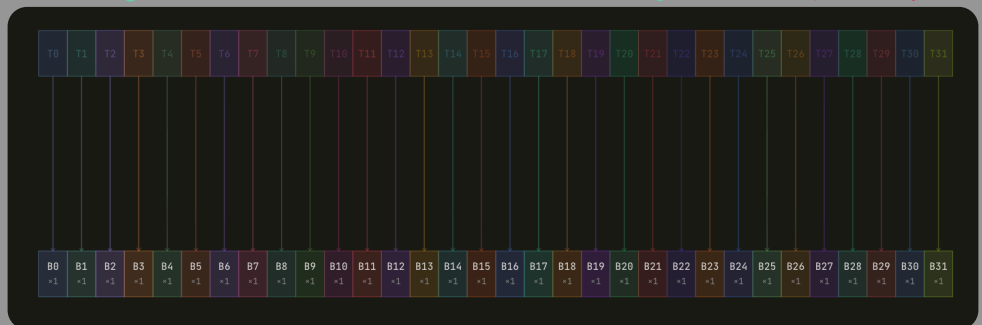
→ Bank Mapping

⇒ Stride patterns

> stride 1

- T_i reads $i \times 4$
- every thread hits a unique bank
- All 32 in one cycle

1 cycle / none conflict / 32 active banks / 100% eff.



! 2 → 2 cycles / 2x conflicts / 16 active banks / 50% eff.

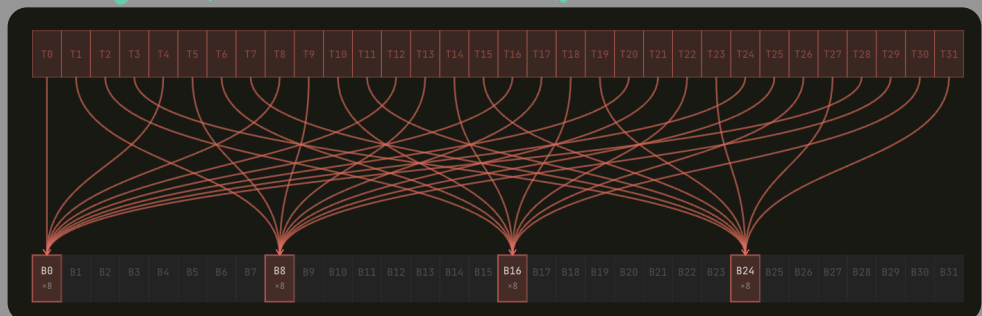
! 4 → 4 cycles / 4x conflicts / 8 active banks / 25% eff.

!

> stride 8

- only 4 banks active, each hit by 8 T
- rest of the array sits idle

8 cycles / 8x conflict / 4 active banks / 12% eff.



! 16 → 16 cycles / 16x conflicts / 2 active banks / 6% eff.

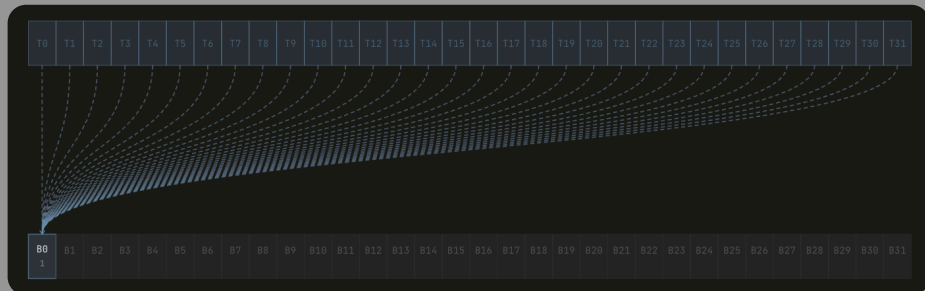
1 32 $\rightarrow$ 32 cycles / 32x conflicts / 1 active banks / 3% eff.

$\Rightarrow$ Special cases

Broadcast: zero cost

All 32 threads read the exact same address. The hardware detects uniform access and serves all threads in one fetch, broadcasting the result to every thread simultaneously. Same cost as stride-1.

1 cycles / none conflict / 1 active banks / 100% efficiency

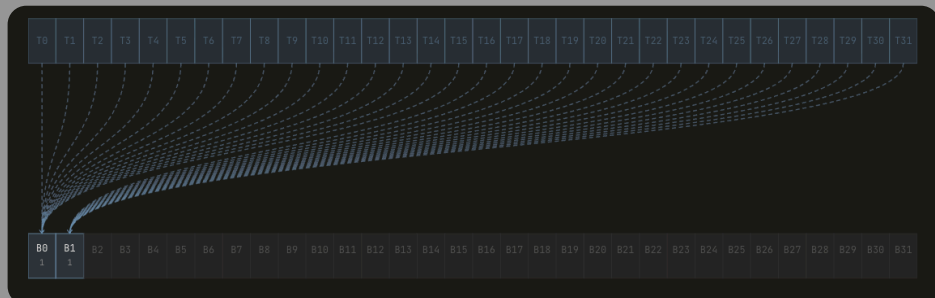


> one fetch, result broadcast to every thread, same cost as stride-1

Multicast (2 groups): zero cost

Two groups of 16 threads each read a different address. Each group gets a multicast, one fetch per unique address. Both fetches land on different banks and happen in parallel. Still one cycle total.

1 cycles / none conflict / 2 active banks / 100% efficiency

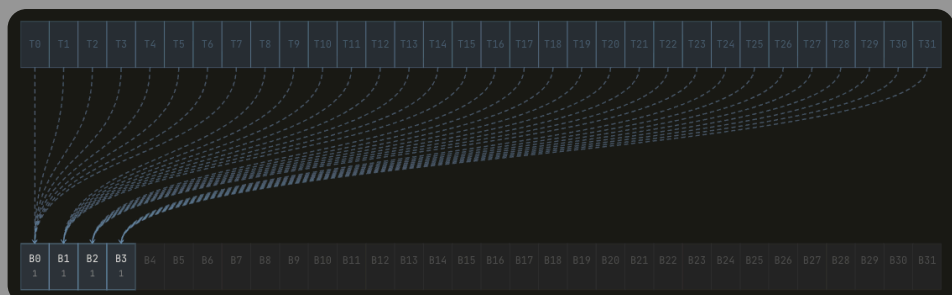


> two addresses on two banks; two fetches in parallel, 1 cycle

Multicast (4 groups): zero cost

Four groups of 8 threads, each reading a different address on a different bank. Four parallel multicasts, each served simultaneously. One cycle total.

1 cycles / none conflicts / 4 active banks / 100% efficiency

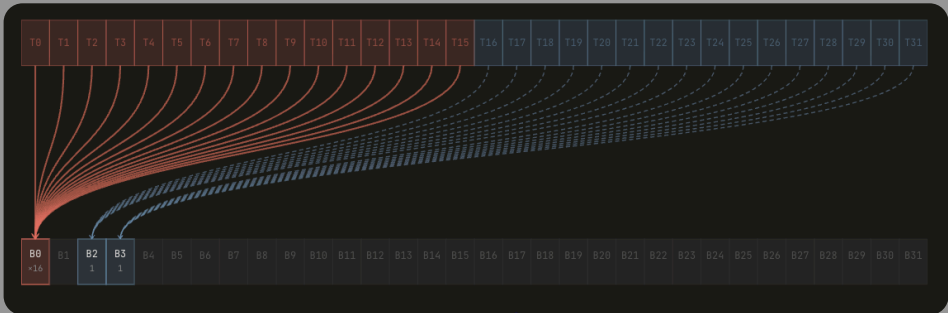


> 4 addresses on 4 banks; 4 multicasts in 1 cycle

Mixed: multicast meets conflict

Groups share an address within the group (free multicast), but two groups map to the same bank with different addresses. That bank still has to serialize. Shows exactly where multicast stops helping.

2 cycles / 2x conflicts / 3 active banks / 50% efficiency

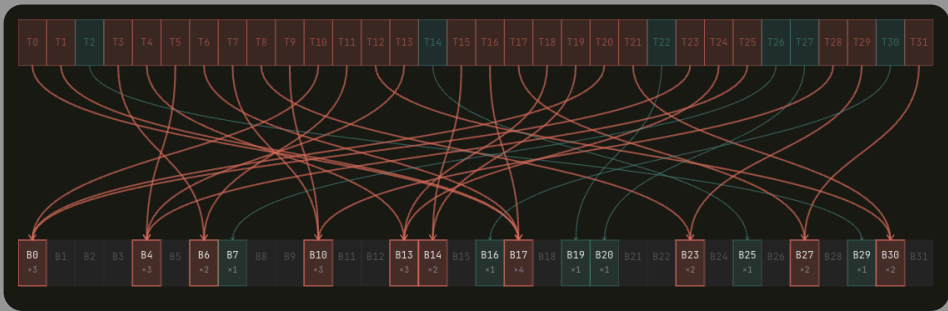


>multiclass within a group is free; diff. addresses on the same bank still serialize

Random: unpredictable

Threads access arbitrary addresses. Conflict degree equals the worst-case bank load, fully unpredictable. A "re-roll" button regenerates the random assignment so you can see how variance plays out. Average conflict is ~1.4x across many rolls.

4 cycles / 4x conflict / 16 active banks / 25% efficiency



> replay count follows the hottest bank.
Re-roll to sample another layout.

→ The Solutions

⇒ Padding

BANK CONFLICT

bank = (row * width + col) % 32
width = 8

T0	0x8+2	→	b2
T1	1x8+2	→	b10
T2	2x8+2	→	b18
T3	3x8+2	→	b26
T4	4x8+2	→	b2
T5	5x8+2	→	b10
T6	6x8+2	→	b18
T7	7x8+2	→	b26

8-way conflict: all threads → bank 2. 8 serialized passes.

b0	b1	b2	b3	b4	b5	b6	b7
b8	b9	b10	b11	b12	b13	b14	b15
b16	b17	b18	b19	b20	b21	b22	b23
b24	b25	b26	b27	b28	b29	b30	b31
b0	b1	b2	b3	b4	b5	b6	b7
b8	b9	b10	b11	b12	b13	b14	b15
b16	b17	b18	b19	b20	b21	b22	b23
b24	b25	b26	b27	b28	b29	b30	b31

> Pad each stream row so row stride widens:

- bank indices shift
- a col read can land on distinct banks i.e. one hot banks

full bandwidth,
at the cost of a lil extra
shmem

NO BANK CONFLICT

bank = (row * width + col) % 32
width = 9

T00×9+2→b2

T11×9+2→b11

T22×9+2→b20

T33×9+2→b29

T44×9+2→b6

T55×9+2→b15

T66×9+2→b24

T77×9+2→b1

No conflict: banks 2, 11, 20, 29, 6, 15, 24, 1, all unique. 1 pass.

b0	b1	b2	b3	b4	b5	b6	b7	padding
b9	b10	b11	b12	b13	b14	b15	b16	
b18	b19	b20	b21	b22	b23	b24	b25	
b27	b28	b29	b30	b31	b0	b1	b2	
b4	b5	b6	b7	b8	b9	b10	b11	
b13	b14	b15	b16	b17	b18	b19	b20	
b22	b23	b24	b25	b26	b27	b28	b29	
b31	b0	b1	b2	b3	b4	b5	b6	

⇒ Swizzling

BANK CONFLICT

Logical read: column 2

T0smem[0][2]→b2

T1smem[1][2]→b10

T2smem[2][2]→b18

T3smem[3][2]→b26

T4smem[4][2]→b2

T5smem[5][2]→b10

T6smem[6][2]→b18

T7smem[7][2]→b26

Column read stacks threads on the same banks → multiple replay passes.

b0	b1	b2	b3	b4	b5	b6	b7
b8	b9	b10	b11	b12	b13	b14	b15
b16	b17	b18	b19	b20	b21	b22	b23
b24	b25	b26	b27	b28	b29	b30	b31
b0	b1	b2	b3	b4	b5	b6	b7
b8	b9	b10	b11	b12	b13	b14	b15
b16	b17	b18	b19	b20	b21	b22	b23
b24	b25	b26	b27	b28	b29	b30	b31

> Linear col. reads stack
threads on the same banks
- XOR swizzle renops
cols per row

so the same logical
col. can hit distinct banks;
full bandwidth
w/out padding

NO BANK CONFLICT

smem[row][col @ (row % 8)]

T02 @ (0 % 8) = 2 → b2

T12 @ (1 % 8) = 3 → b11

T22 @ (2 % 8) = 0 → b16

T32 @ (3 % 8) = 1 → b25

T42 @ (4 % 8) = 6 → b6

T52 @ (5 % 8) = 7 → b15

T62 @ (6 % 8) = 4 → b20

T72 @ (7 % 8) = 5 → b29

No conflict: same eight cells (logical column 2), now eight unique banks. One pass, no padding.

b0	b1	b2	b3	b4	b5	b6	b7
b8	b9	b10	b11	b12	b13	b14	b15
b16	b17	b18	b19	b20	b21	b22	b23
b24	b25	b26	b27	b28	b29	b30	b31
b0	b1	b2	b3	b4	b5	b6	b7
b8	b9	b10	b11	b12	b13	b14	b15
b16	b17	b18	b19	b20	b21	b22	b23
b24	b25	b26	b27	b28	b29	b30	b31